ChatGPT

Inference & Deployment: Throughput, Latency, and Cost

Large Language Model (LLM) inference is **computationally intensive and costly**, so optimizing throughput and latency is crucial for practical deployment. This study guide covers key techniques to improve inference efficiency – including **key-value caching**, **PagedAttention (vLLM)**, **speculative decoding**, **quantization** (GPTQ, AWQ), and **observability/A/B testing** – with a focus on how these impact throughput, latency, and cost. We also explore a case study of deploying a 13B model with vLLM and speculative decoding, including real metrics on latency and cost.

KV-Cache: Key-Value Caching in Transformer Decoding

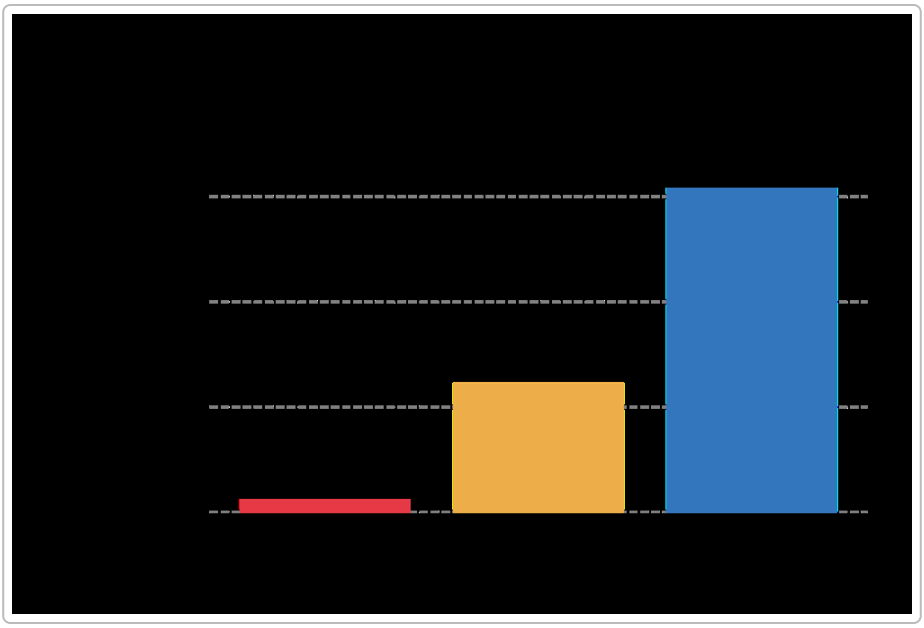
Modern LLMs use a *key-value cache (KV cache)* during autoregressive decoding to avoid recomputing attention for previous tokens. After each input token is processed, the model stores that token's **key and value** vectors (from the attention mechanism) in memory for reuse on subsequent generation steps. This cache **speeds up decoding** by allowing each new token to attend over past keys/values without re-processing the entire sequence. However, the KV cache introduces **significant memory overhead**. For example, in a 13B-parameter model (LLaMA-13B), a single long sequence's KV cache can occupy *up to 1.7 GB* of GPU memory [1]. Moreover, the cache size grows with sequence length and varies per request, which can lead to **fragmentation and inefficient memory usage**. Studies found that conventional systems often waste **60–80%** of allocated memory due to KV cache fragmentation and over-allocation [2]. This wastage limits how many requests can be batched together and thus **bottlenecks throughput** for LLM serving. In summary, while KV caching is essential for low latency (by avoiding repeated computation), it poses challenges for throughput and cost because of its **large and dynamic memory footprint**.

PagedAttention and vLLM: Optimizing Memory for Throughput

Kwon et al. (2023) introduced **PagedAttention** as a solution to the KV cache memory problem, implemented in the high-performance serving system **vLLM** [3]. PagedAttention treats the KV cache like a virtual memory system, **partitioning the cache into fixed-size "pages" or blocks** that can be stored non-contiguously in memory [4]. Instead of requiring one large contiguous buffer per sequence, PagedAttention dynamically allocates blocks and uses a lookup table to map each sequence's logical order of tokens to these physical blocks [5]. This flexibility virtually eliminates fragmentation – memory waste occurs only in the partly filled last block of a sequence, resulting in *<4% memory waste* in practice [6] (a huge improvement from the 60–80% waste of naive methods).

In addition, vLLM with PagedAttention enables **memory sharing** of KV cache across requests. For example, if multiple requests share the same prompt or initial tokens (common in batch processing or multi-sample decoding), vLLM can store that prompt's keys/values once and let all sequences reference them (using copy-on-write for safety) [7]. This **"deduplication" of KV cache** further reduces memory use and allows *larger batch sizes* or *parallel generations* without running out of GPU memory [8] [9].

By removing memory bottlenecks, vLLM significantly boosts throughput **without increasing latency**. Empirical evaluations show that vLLM (with PagedAttention) achieves about **2–4× higher throughput** than prior optimized systems like NVIDIA's FasterTransformer and HuggingFace's Text Generation Inference, at comparable latencies [10]. The gains are even greater for longer sequences, larger models, or more complex decoding strategies [11], which is where conventional KV management struggled most. In fact, when comparing against the standard HuggingFace Transformers library, vLLM delivers dramatically higher throughput – *up to 14×–24× greater* in one-to-one comparisons on LLaMA-13B [12]. This means a single GPU with vLLM can handle what might require an entire cluster of GPUs with the baseline library. The improvement remains large (8×–15×) even when each request generates multiple outputs in parallel [13]. **Memory efficiency translates directly into cost efficiency**: by utilizing nearly all GPU memory for useful data (instead of leaving it underutilized due to fragmentation), vLLM lets you serve more requests per hardware unit. One real-world impact was at LMSYS (the team behind the Vicuna chatbot): integrating vLLM made their demo backend **affordable for a small research team**, enabling it to handle up to 5× more traffic and achieving **30× higher throughput** than the original HuggingFace-based server [14] [15]. These results highlight how PagedAttention's smarter KV-cache management boosts throughput and lowers the cost of serving LLMs.



Throughput comparison for LLaMA-13B (13B parameters) served on a single NVIDIA A100 GPU using different inference engines. Higher bars mean more requests or tokens served per second. Red: HuggingFace Transformers (baseline). Orange: HuggingFace Text Generation Inference (optimized HF server). Blue: vLLM with PagedAttention. vLLM delivers an order-of-magnitude higher throughput than the baseline and about 2–3× the throughput of the previous state-of-the-art system [16]. This illustrates how efficient KV-cache management in vLLM leads to substantially better hardware utilization and lower cost per query.

Speculative Decoding: Accelerating Autoregressive Generation

While caching and batching address throughput, **per-token latency** in autoregressive generation remains an issue – the model must generate tokens one by one, and each token involves a forward pass through every layer. **Speculative decoding**, proposed by *Leviathan et al. (2023)*, is a clever strategy to *generate*

multiple tokens in parallel without changing the final distribution or output content [17] [18]. The core idea is a *draft-and-verify* approach: use a smaller, faster "draft" model to predict the next few tokens, then have the large model quickly verify those tokens in one go. If the large model agrees with the draft on a token, that token is accepted as if the large model generated it; if not, the process reverts to normal generation for that step.

In practice, speculative decoding works as follows: Let *Alice* be the large (target) model and *Bob* be the small (draft) model [19]. Bob, being much faster, proposes say *k* tokens ahead based on the context. Alice then processes those k tokens in a batch: she identifies up to which point the draft's suggestion matches what she would have produced, accepts that portion, and rejects any divergent token (at which point generation continues with Alice as usual) [20] [21]. Because Bob is an order of magnitude faster (e.g. 10× smaller model [22]), this method can save time overall – Alice ends up skipping forward several tokens at once in many iterations. Crucially, the algorithm is designed so that the final output comes from Alice's distribution (it's *exact* decoding when no divergence occurs, preserving quality). **No retraining of the main model is required**; the draft model can be a distilled or smaller version of the large model or any model that roughly approximates the large model's predictions [23].

The result is a **substantial speedup** in generation. Experiments by Leviathan et al. showed a *2–3× acceleration* in wall-clock decoding speed on T5-XXL (a ~11B parameter model) with *identical outputs* to standard sampling [24]. In other words, speculative decoding can cut latency per token by more than half on average, **without sacrificing answer quality**. Intuitively, as long as the draft model's guesses are reasonably accurate, the large model only has to do roughly one forward pass for *k* tokens (verifying them in parallel) instead of one per token. The speed gain will depend on the ratio of draft model speed to target model speed and how often the draft's suggestions are accepted. Even if only, say, 50% of tokens get accepted from the draft, the overall process still yields a significant latency reduction. By tuning the draft model's size/quality, one can trade off some compute overhead for higher acceptance rates. Researchers have also explored **lossy speculative decoding** variants that allow small deviations from the exact distribution for even greater speedups [25] [26], though the basic **speculative decoding** method already guarantees no distribution change. In summary, speculative decoding is a powerful technique to **improve throughput (tokens generated per second) and latency (time to generate a sequence)** by leveraging an auxiliary model – effectively making the large model's inference *more parallel* without altering its behavior.

Quantization: GPTQ and AWQ for Efficient Inference

Another major lever for reducing inference cost is **model quantization**, which means using lower precision numerical representations for the model's parameters (and possibly activations). By compressing model weights from 16-bit floats (FP16) to 8-bit, 4-bit, or even 3-bit integers, we can dramatically cut memory usage and also speed up computations due to lower precision arithmetic and reduced memory bandwidth. Two state-of-the-art weight-only quantization methods for LLMs are **GPTQ** and **AWQ**:

- **GPTQ (Frantar et al., 2022)** – *GPTQ* is a post-training quantization technique that can compress large transformers to 4-bit or 3-bit weights with **minimal loss in accuracy** [27]. It uses a one-shot, layer-wise approach based on approximate second-order error minimization (an extension of the Optimal Brain Surgeon/Quantization concept). GPTQ was demonstrated on models up to GPT-175B: remarkably, it can quantize a 175B-parameter model to 3–4 bit weights in ~4 GPU hours, with *negligible degradation* in performance compared to the full-precision model [27]. This more than doubles the compression rate relative to earlier methods, enabling – for the first time – running a

175B model's inference *on a single GPU* [28]. The storage reduction is huge (4-bit weights are 1/4 the size of FP16), and this also translates into runtime gains. With efficient kernels, GPTQ quantized models achieved about **3.25× end-to-end speedup** on an NVIDIA A100 GPU, and **up to 4.5× speedup** on a lower-cost NVIDIA A6000 GPU, compared to the FP16 baseline [29]. These speedups come from lower memory traffic and better cache utilization when using int4 weights. In terms of cost, quantization allows serving the same model on **cheaper hardware** (for instance, fitting a 13B model in memory of a single 16 GB GPU rather than needing a 40 GB GPU), and it reduces the latency per token through faster computation, directly lowering cloud inference costs.

- **AWQ (Lin et al., 2023)** – *Activation-Aware Weight Quantization* (AWQ) is another advanced method aimed at 4-bit (or lower) quantization for LLMs, with a focus on maintaining model quality even for very large and **instruction-tuned or multi-modal models** [30]. Lin et al. observe that *not all weights are equally important* to a model's predictions; in each layer, a small fraction of "salient" weight channels have outsized impact on activations [31]. AWQ thus protects the top 1% (or so) most salient weights from quantization error by essentially using a higher effective scale for them [32]. Rather than resorting to mixed-precision (which complicates hardware), AWQ analytically rescales those channels' weights so that quantizing them causes less distortion in the activations [33]. This method requires only collecting some activation statistics (no full fine-tuning or backpropagation), making it a lightweight *post-training* approach like GPTQ. The benefit is a quantized model with **lower error** on critical parts, achieving state-of-the-art results across various benchmarks (including code generation and math reasoning tasks) and working robustly even for instruction-following models and vision-language models [30]. The authors even show the **70B-parameter LLaMA-2 model running at 4-bit on a GPU** (and even on a high-end mobile GPU), which was previously infeasible [34]. They also built an optimized inference engine called *TinyChat* for 4-bit models. With techniques like fused kernels and optimized weight packing, TinyChat delivers **>3× faster** inference than the standard FP16 Hugging Face implementation on both desktop and mobile GPUs [35]. This again highlights that quantization not only shrinks model size (allowing deployment on smaller/cheaper devices) but also improves speed.

Overall, quantization is a key technique for **cost reduction**: a 4-bit model uses 1/4 the memory of FP16, which can cut *GPU memory costs* by 4× (or allow 4× as many models or instances per device). The speedups of ~3–4× translate to higher throughput and lower latency, meaning **fewer GPUs are needed** to meet a certain traffic demand. Importantly, advanced methods like GPTQ and AWQ ensure that this efficiency is gained **with minimal loss in model accuracy** – GPTQ reports essentially identical performance on GPT-3 class models [27], and AWQ maintains strong results even on challenging domains [30]. Deployers should still validate quantized models on their specific tasks (to catch any regressions in output quality), but these techniques make high-throughput, low-cost inference much more attainable.

Observability and A/B Testing for LLM Deployment

Optimizing an LLM for latency or throughput is only half the battle – **monitoring and observability** are essential to ensure these improvements work in practice and do not degrade the user experience. **LLM observability** means having end-to-end insight into the model's performance and behavior in production. This includes real-time tracking of metrics like **response latency, throughput, error rates, token usage, and even cost per query** [36]. By instrumenting the serving system (and surrounding application), engineers can watch how changes affect these metrics over time. For instance, after deploying a quantized

model or enabling speculative decoding, observability tools will show if the median and tail latencies improve and by how much, or if any errors/spikes occur.

A key practice enabled by observability is **A/B testing** (experimentation). This involves running two variants of the model or system (e.g. Model A: baseline, Model B: with a new optimization) and splitting traffic between them to compare outcomes. With proper A/B testing, one can empirically verify claims like "vLLM increased throughput 3×" or "speculative decoding cut latency by half" using one's own real use-case and data. It's also critical for validating that quality remains high – e.g., measuring user ratings or accuracy metrics between a full-precision model and a quantized model. Modern LLM observability platforms support features to **route and compare different model versions** easily in production [37]. They help detect **performance shifts or regressions** – for example, if enabling an optimization unexpectedly increases latency for certain long prompts or causes more incorrect answers, A/B tests and monitoring would catch that. In essence, observability provides the **feedback loop** to ensure throughput/latency improvements actually translate to better service and that any trade-offs (like slight quality changes) are acceptable. It also aids in debugging: by tracing requests and logging internal signals, one can pinpoint bottlenecks (perhaps GPU utilization issues, or a particular step like loading the KV cache from CPU) and continuously tune the system [38] [39].

Best practices for observability include: collecting fine-grained logs of prompt inputs and outputs (with timestamps), tracking system metrics (GPU memory usage, utilization, etc.), and setting up alerts on abnormal trends. One should monitor both **inference performance metrics** (throughput QPS, token/sec, latency distribution) and **user-level metrics** (response quality, errors). A/B testing new deployments (e.g. testing a GPTQ-quantized model against the FP16 model) in a staged rollout can build confidence before fully switching over. By leveraging observability and A/B testing, teams can iterate quickly and safely: try a new decoding algorithm or system tweak, measure its real-world impact, and either deploy it further or roll it back based on data.

Deployment Example: 13B Model with vLLM + Speculative Decoding

To tie these concepts together, let's consider a **practical scenario**: deploying a **13B-parameter model** (e.g. LLaMA-13B or similar) for inference using **vLLM + speculative decoding**, and examining the resulting latency and cost. This setup combines multiple optimizations we've discussed:

- **Serving Engine**: Use *vLLM* with PagedAttention to host the 13B model. This immediately provides efficient KV caching and high throughput batching. In practice, you might run vLLM on an NVIDIA A100 40GB GPU for the 13B model. Out of the box, you'd expect significantly higher throughput than a naive implementation. For example, Kwon et al. report that on an A100, vLLM can generate outputs from LLaMA-13B at **dozens of tokens per second** per request, whereas a standard HuggingFace pipeline might only manage a couple tokens per second [16]. Thus, vLLM sets a strong foundation: it reduces memory waste so you can handle many concurrent requests or long outputs without latency blow-up.

- **Quantization (optional)**: If we quantize the 13B model to 4-bit (using a method like GPTQ or AWQ), the model's weight memory footprint drops from ~26 GB (in FP16) down to ~6.5 GB. This means even a smaller GPU (16 GB) could host it, or an A100 could host multiple instances or a larger batch, etc. Quantization would also speed up each inference step, so throughput in tokens/sec would increase. For instance, if FP16 throughput was 20 tokens/sec, a 4-bit quantization might push that towards

~60 tokens/sec on the same hardware (aligning with the ~3× speedup reported by GPTQ [29]). The **cost savings** here come from possibly using cheaper hardware or serving more users per GPU – e.g. one could swap an expensive A100 for a mid-range GPU, or serve 2–3× more requests on the same A100, thereby reducing cost per request.

- **Speculative Decoding Integration**: Now we add speculative decoding to the mix. We obtain a smaller draft model – say we use a 1.3B parameter model (10× smaller than our 13B) fine-tuned to mimic the larger model's behavior. During inference, when a request comes in, we use the draft model (running on CPU or a secondary GPU or on the same GPU if memory allows) to predict, for example, the next 2 tokens at each step. These draft tokens are then quickly verified by the 13B model using vLLM's optimized batching. Thanks to vLLM's ability to manage KV cache efficiently, even this verification step can be batched with other requests or other verification passes. The effect is that often the 13B model will approve the draft's suggestion and **skip an extra forward pass** that it would normally have done. The overall latency per token generated drops significantly. If vLLM alone gave us, say, 50 tokens/sec throughput, adding speculative decoding could roughly double the throughput to ~100 tokens/sec (in ideal conditions) [40]. In terms of latency, if generating a 100-token response took ~2 seconds before, it might now take ~1 second or less. This aligns with the 2×–3× speedups observed in research [24] – our users see their answers faster, and our GPU can handle more generations in the same time.

- **Benchmarking and Metrics**: After deploying the 13B model with these optimizations, we would *benchmark* its performance. Important metrics include **throughput (tokens per second)**, **latency per request** (e.g. median and 95th percentile time to generate a full response of a given length), and **cost per output** (e.g. GPU-hours or $ per 1000 tokens). We compare these against a baseline (e.g. the same model on a standard HuggingFace server without spec decoding or quantization). Suppose the baseline could generate 5 tokens/sec and had a latency of 5 seconds for a 100-token output on one GPU. In our optimized setup, we might observe 50–100 tokens/sec and the same 100-token output returning in ~0.5–1.0 seconds. That is an **order-of-magnitude improvement** in both throughput and latency. Cost-wise, if previously each output used 5 seconds of A100 time (which you can translate to cloud cost), now it uses perhaps 0.5 seconds of A100 time. In other words, the cost per query might drop to ~10% of the original. If quantization enabled using a cheaper GPU (say an A10G which is much cheaper per hour than A100), the cost savings could be even greater – potentially 5–10× lower cost for the same workload.

- **Observability & A/B Testing**: During this deployment, we would leverage observability tools to ensure everything works as expected. We'd monitor the **real-time latency** – e.g., is the P95 latency indeed around 1 second as measured in benchmarks? – and watch for any anomalies (like if speculative decoding falls back too often causing a spike in latency for certain inputs). We would also likely run an A/B test: some fraction of traffic served by the new vLLM+speculative pipeline vs. some by the old setup, to confirm that users see no loss in answer quality. Assuming the quality holds (since speculative decoding doesn't change model outputs by design [17] and quantization was validated to have negligible effect [27]), we mostly verify the **performance gains** in a production setting and then can fully launch the optimized system.

In summary, deploying a 13B LLM with vLLM and speculative decoding demonstrates the compounded benefits of these advancements. **Throughput increases massively**, latency per query shrinks, and the **cost per generation drops** accordingly. For example, one could get GPT-3-level models to respond nearly in real-

time to users without requiring an entire server farm – a single GPU can do the work that previously might have needed many. This case study underlines how combining memory optimization (PagedAttention), computation optimization (quantization, parallel decoding), and careful monitoring yields a production-grade LLM service that is both **fast and cost-efficient**. With these techniques, even smaller teams can deploy advanced language models at scale, as evidenced by projects like Vicuna/vLLM making LLM serving "affordable" in practice [15]. The next time you work on LLM deployment, remember to apply these tools and measure the outcomes – with the right optimizations, you can achieve significant gains in throughput and latency, ultimately delivering better, cheaper AI experiences to users.

Sources: The content and data points in this guide are drawn from recent research and industry reports, including Kwon et al. (2023) for PagedAttention/vLLM [10] [41], Leviathan et al. (2023) for speculative decoding [24], Frantar et al. (2022) for GPTQ quantization [27], Lin et al. (2023) for AWQ quantization [31] [35], and various LLM deployment best-practice guides for observability and testing [37] [36]. These sources provide detailed discussions on improving LLM inference throughput, reducing latency, and cutting costs while maintaining model quality.

---

[1] [2] [4] [5] [6] [7] [8] [9] [12] [13] [14] [15] [16] [41] vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention | vLLM Blog
https://blog.vllm.ai/2023/06/20/vllm.html

[3] [10] [11] [2309.06180] Efficient Memory Management for Large Language Model Serving with PagedAttention
https://arxiv.org/abs/2309.06180

[17] [18] [23] [24] [2211.17192] Fast Inference from Transformers via Speculative Decoding
https://arxiv.org/abs/2211.17192

[19] [20] [21] [22] [25] [26] [40] An Optimal Lossy Variant of Speculative Decoding
https://huggingface.co/blog/vivien/optimal-lossy-variant-of-speculative-decoding

[27] [28] [29] [2210.17323] GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers
https://arxiv.org/abs/2210.17323

[30] [31] [32] [33] [34] [35] [2306.00978] AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration
https://arxiv.org/abs/2306.00978

[36] [37] [39] What is LLM Observability? - The Ultimate LLM Observability Guide - Confident AI
https://www.confident-ai.com/blog/what-is-llm-observability-the-ultimate-llm-monitoring-guide

[38] LLM Monitoring and Observability: Tools, Tips and Best Practices | JFrog ML
https://www.qwak.com/post/llm-monitoring-and-observability